Now write the php code to connect to the database

In this example the object-oriented connection style will be explained. **Note that the procedural interface starting with "mysql_" (like mysql_connect) is deprecated!**

```php
<?php

SESSION_START();

// mySQL database information
$_db_host = "db_server";
$_db_datenbank = "web";
$_db_username = "web";
$_db_passwort = "database-password";



# open database connection
$conn = new mysqli($_db_host, $_db_username, $_db_passwort, $_db_datenbank);



// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// TODO: Here database is connected use it for Select, Insert, ....

# close database
$conn->close();

?>
```

1. Information about **hostname**, **database**, **mySQL-user** and **password** are needed to connect to mySQL.

2. Open the connection by creating an object of the class "mysqli".

3. With the new object the connection status can be checked.

4. Now the mySQL database connection can be used. (An example will be shown below.)

5. After using the database the connection must be closed!

Figure 1: First MySql-php code to connect to the database.

Hint: If you want to use one single file in your application that creates the DB-connection it is a good idea to have one single „database.php" file that includes the above code without the closing of the connection and use the $conn object outside after requiring the database.php file.

```php
<?php
require_once "database.php";

// TODO: Here database is connected. Use $conn for Select, Insert, …

$conn->close();
```

Now we add PHP code to add a new user to the database

First we create an HTML file where new user infromation can be inserted within a form.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Create new User</title>
</head>
<body>
<form method="POST" action="createUserMySql.php">
    New username: <input name="username"><br>
    Password: <input name="passwort1" type=password><br>
    Retype password: <input name="passwort2" type=password><br>
    <input type=submit name=submit value="Create new user">
</form>
</body>
</html>
```

This form will call the PHP file createUserMySql.php on submit.

Figure 2: HTML file "create_user_form.html" to input new user details.

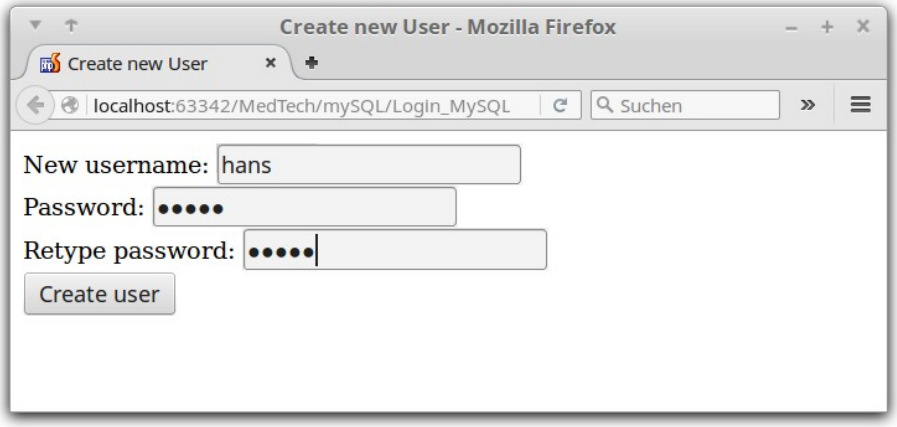

Figure 3: Userform for entering a new user.

Now we have a form which will be able to get new user details. Next we have to enhance our PHP program to check the password and try to add the new user in the database. As the column username in the database ist declared “unique” it automatically checks if the user already exists.

Enhance PHP-code in the file “createUserMySql.php” to write the user to db

```php
# open database connection
$conn = new mysqli($_db_host, $_db_username, $_db_passwort, $_db_datenbank);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (!empty($_POST["submit"])) {
    # get data from POST array and parse database special characters.
    # this is needed to block hacking attacks
    $_username = $conn->real_escape_string($_POST["username"]);
    $_passwort = $conn->real_escape_string($_POST["passwort1"]);
    if (strcmp($_passwort, $conn->real_escape_string($_POST["passwort2"])) != 0) {
        # password is not repeated correctly
        include("create_user_form.html");
        exit;
    }

    # create password hash from original password
    # VARCHAR 60 necessary, but officially PHP recommendation: at least 255 characters
    $_passwortHash = password_hash($_passwort, PASSWORD_BCRYPT);


    # Statement for insert the values of the new user
    $insertStatement = "INSERT INTO login_username (username, password, user_deleted, last_login)
                        VALUES ('$_username', '$_passwortHash', 0, NOW());";

    if ($_res = $conn->query($insertStatement)) {
        echo "<br>User $_username has been added to the database.<br>Try to log in.";
        include("login_form.html");
    } else {
        echo "<br>NO insertion. User could not be added. Maybe user $_username already exists.";
        include("create_user_form.html");
    }

} else {
    include("create_user_form.html");
}

# close database
$conn->close();
?>
```

1. Check if data is available from the html-form.

2. Check if data is available from the html-form and if password is valid.

3. This makes the md5 value of the password more secure.

4. Create the insert-SQL statement using the form input.
The hash-function from PHP will store the password hash not the plain text!

5. Run the query to insert the new user and check if the user details could be stored in the table.

6. On Error show the “create_user_form” again.
Otherwise show the “login_form”

Figure 4: File "createUserMySql.php"

Now open the HTML-form “create_user_form.html” and fill with correct data. By pressing the “Create user” button the new user should be stored in the database.
(Check this with phpMyAdmin!)

Now we create a HTML-form to login as the new user

This is similar as the first form but we do not need to fields for the password. And the button will be used for Login

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title></title>
</head>
<body>
<form method="POST" action="loginMySql.php">
    Username: <input name="username"><br>
    Password: <input name="passwort" type=password><br>
    <input type=submit name=submit value="Login">
</form>
</body>
</html>
```

This form will call the php file “loginMySql.php” on submit.

Figure 5: File "login_form.html" for entering the login data of the user.

Now we create PHP code to login as a user

```php
<?php

SESSION_START();

// mySQL database information
$_db_host = "db_server";
$_db_datenbank = "web";
$_db_username = "web";
$_db_passwort = "database-password";



# open database connection
$conn = new mysqli($_db_host, $_db_username, $_db_passwort, $_db_datenbank);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (!empty($_POST["submit"])) {
    # get data from POST array
    $_username = $_POST["username"];
    $_passwort = $_POST["passwort"];

    /* User from db */
    $stmt = $conn->prepare(
        "SELECT * FROM login_username WHERE username = ? AND user_deleted = 0 LIMIT 1"
    );
    $stmt->bind_param("s", $_username);
    $stmt->execute();

    $res = $stmt->get_result();
```

1. Use a session for login.

2. Get the formular data.

3. Prepare a select statement for the database query.
? is a placeholder for name.

4. Bind value username to select statement.
(s = String/varchar)

Figure 6: First part of file "loginMySql.php"

```
34
35      if ($res->num_rows === 1) {
36          $user = $res->fetch_assoc();
37
38          /* Passwort prüfen */
39          if (password_verify($_passwort, $user["password"]))
40
41              $_SESSION["login"] = 1;
42              $_SESSION["user"]  = $user;
43
44              /* last_login aktualisieren */
45              $stmt = $conn->prepare(
46                  "UPDATE login_username SET last_login = NOW() WHERE id = ?"
47              );
48              $stmt->bind_param("i", $user["id"]);
49              $stmt->execute();
50
51
52
53          } else {
54              echo "Passwort falsch.<br>";
55              include("login_form.html");
56          }
57      } else {
58          echo "Benutzer nicht gefunden.<br>";
59          include("login_form.html");
60      }
61  }
62
63  # close database
64  $conn->close();
65
66  # Is user already logged in ???
67
68
69  if (is_array($_SESSION) && isset($_SESSION["login"]) && $_SESSION["login"] == 1) {
70
71
72      # Todo: Add program code for logged in user !!
73      header("Location: secretContent.php");
74  }
75
76
77  ?>
```

5. Check the select result. If there is a row then the user has been found.

6. Do password verification and set some session variables.

8. Set the new last login time in the database.

9. If the login doesn't succeed show the login formula again.

11. If the user is not known show the login formula again.

12. If SESSION has valid login you can show the user his secret information. (No echo is allowed before!)

Figure 7: Second part of file "loginMySql.php"

secretContent.php

```php
<?php
require_once "auth_check.php";

$username = htmlspecialchars($_SESSION["user"]["username"], ENT_QUOTES, 'UTF-8');
?>
<h1>Geheimer Bereich</h1>

<p>Willkommen <?php echo $username; ?></p>

<p>
    <a href="logout.php">Logout</a>
</p>
```

auth_check.php

```php
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

/* check login */
if (
    empty($_SESSION["login"]) ||
    $_SESSION["login"] !== 1 ||
    empty($_SESSION["user"])
) {
    /* Nicht eingeloggt */
    header("Location: login_form.html");
    exit;
}

/* Optional: Session-Fixation-Schutz */
session_regenerate_id(true);
```

logout.php

```php
<?php
session_start();

/* delete all values */
$_SESSION = [];

/* Session-cookie should be deleted (!) */
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(
        session_name(),
        '',
        time() - 42000,
        $params["path"],
        $params["domain"],
        $params["secure"],
        $params["httponly"]
    );
}


session_destroy();

/* Forward to Login */
header("Location: login_form.html");
exit;

?>
```